

Unity FPS Pick and Drop Guide

Planet App / FPS Project

Unity FPS Pick and Drop - Complete Setup Guide

This guide walks through building a working “press E to pick up, press Q to drop” interaction system in an FPS Unity project, using a component-based marker (`Pickupable`) instead of Unity’s Tag system, since string tags proved unreliable in this project (intermittent “Tag not defined” errors).

Step 1: Create the Unity Project

1. Open **Unity Hub** → **New Project**.
 2. Choose the **Universal 3D (URP)** template.
 3. Name it (e.g. `S_Content_Overview`) and click **Create**.
-

Step 2: Get an FPS Template

1. In the editor, open **Window** → **Asset Store** (or browse assetstore.unity.com in Chrome).
 2. Search for an FPS template - e.g. **Infima Games’ Low Poly Shooter Pack**.
 3. Click **Download**, then **Import** - accept all files in the import dialog.
-

Step 3: Open the Demo Scene

In the **Project** panel, navigate to:

```
Assets → Infima Games → Low Poly Shooter Pack → Scenes
```

Double-click the demo scene to open it.

Step 4: Prep Pickable Objects with Physics

For each object you want pickable (barrels, gas tanks, etc.):

1. Select it in the Hierarchy.
 2. **Add Component** → **Rigidbody**.
 3. **Add Component** → **Box Collider** (skip if it already has one).
-

Step 5: Set Active Input Handling

This avoids a major bug: relying only on the legacy `Input` class silently fails in projects using the new Input System package.

1. **Edit** → **Project Settings** → **Player** → **Other Settings** → **Active Input Handling**.
 2. Set to **“Both”**.
 3. Let Unity restart if prompted.
-

Step 6: Create the HoldPoint

1. In the Hierarchy, find your actual rendering **Camera** (e.g. SOCKET_Camera → Camera).
2. Right-click **Camera** → **Create Empty** → rename it holdpoint .
3. Select holdpoint → set **Transform** → **Position** to roughly (0, 0, 2) (local space) - adjust until it sits naturally in front of the camera view.

Step 7: Create the Pickupable Marker Script

Instead of Unity's Tag system, use a simple component as a marker - more reliable.

1. Right-click in Assets → **Create** → **C# Script** → name it Pickupable .
2. Open it and replace contents with:

```
using UnityEngine;

public class Pickupable : MonoBehaviour
{
}
```

3. Save.

Step 8: Add Pickupable to Every Object You Want Pickable

Select each barrel/gas tank → **Add Component** → search Pickupable → add it. Repeat for every object that should be interactable.

Step 9: Create the Pick-and-Drop Script

1. Right-click in Assets → **Create** → **C# Script** → name it pickanddrop .
2. Open it and replace contents with:

```
using UnityEngine;
using UnityEngine.InputSystem;

public class pickanddrop : MonoBehaviour
{
    public Transform holdPoint;
    private GameObject heldObject;
    private float pickUpDistance = 3f;

    void Update()
    {
        if (Keyboard.current == null) return;

        if (Keyboard.current.eKey.wasPressedThisFrame && heldObject == null)
        {
            TryPickUp();
        }
        else if (Keyboard.current.qKey.wasPressedThisFrame && heldObject != null)
        {
            Drop();
        }
    }

    void TryPickUp()
    {
        Ray ray = Camera.main.ScreenPointToRay(new Vector3(Screen.width / 2, Screen.height / 2, 0));
        RaycastHit hit;

        if (Physics.Raycast(ray, out hit, pickUpDistance))
```

```

    {
        Pickupable pickupable = hit.collider.GetComponent<Pickupable>();

        if (pickupable != null)
        {
            heldObject = hit.collider.gameObject;
            Rigidbody rb = heldObject.GetComponent<Rigidbody>();
            rb.useGravity = false;
            rb.isKinematic = true;
            heldObject.transform.SetParent(holdPoint);
            heldObject.transform.localPosition = Vector3.zero;
            heldObject.transform.localRotation = Quaternion.identity;
        }
    }

    void Drop()
    {
        Rigidbody rb = heldObject.GetComponent<Rigidbody>();
        heldObject.transform.SetParent(null);
        rb.useGravity = true;
        rb.isKinematic = false;
        heldObject = null;
    }
}

```

3. Save.

Step 10: Attach the Script to the Camera

1. Select the **Camera** GameObject (the actual rendering camera, not a parent rig object).
2. **Add Component** → search `pickanddrop` → add it.
3. A **Hold Point** field appears in the Inspector - drag your `holdpoint` GameObject into it.
4. Confirm the Camera's **Tag** dropdown (top of Inspector) is set to `MainCamera`.

Step 11: Play and Test

1. Click **Play**.
2. Use **WASD** to walk toward a pickable object.
3. Aim your crosshair (center of screen) at it.
4. Press **E** - it should snap into place in front of the camera at the `holdpoint` position.
5. Press **Q** - it should detach and fall naturally with gravity restored.

Step 12: Confirm It's Working

Open **Window** → **General** → **Console**, and while playing, you should see no errors - just the object visibly moving in the Game view when you press E/Q.

Troubleshooting Notes (from real debugging)

- **“Tag: X is not defined” errors when using `CompareTag`**: this can happen even when the tag is visibly listed in *Edit* → *Project Settings* → *Tags and Layers*, especially on prefab instances with tag overrides. The `Pickupable` component approach in this guide avoids this class of bug entirely.
- **E key not registering at all**: check *Active Input Handling* is set to “Both” (Step 5). If it's set to only “Input System Package (New)”, legacy `Input.GetKeyDown()` calls silently fail. This guide's script uses the new Input System's `Keyboard.current` directly, which avoids this issue too.

- `Camera.main` returns null / **NullReferenceException**: confirm the actual rendering Camera GameObject has its Tag set to `MainCamera`.
- **Nothing happens and no console output at all**: confirm you are actually in Play mode, and that the Game view has input focus (click inside it once after pressing Play).